



TEXT NORMALIZATION INTERNSHIP CHALLENGE FINAL REPORT

IndabaX report



AUTHOR: NKUIN EUGENE MBENG
21 NOVEMBER 2025

FIGURES

Figure 1 : Google TTS 4

Figure 2: kaldi 4

Figure 3: WER COMPARISON OF FRENCH AND ENGLISH FST 9

Figure 4: English FST 10

Figure 5: French FST 10

Contents

CHAPTER 1: GENERAL INTRODUCTION	3
1.1. BACKGROUND AND MOTIVATION	4
CHAPTER 2: SYSTEM ARCHITECTURE.....	5
1. Unit FST.....	5
2. Teens FST	5
3. Tens FST	5
4. Hundreds FST	6
5. One-Thousand Special Case	6
CHAPTER 3 : METHODOLOGY.....	7
3.1. REQUIREMENT ANALYSIS.....	7
3.2. FST DESIGN	7
3.3. IMPLEMENTATION.....	7
3.4. TESTING	8
3.5. OUTPUT VERIFICATION	8
CHAPTER 4: RESULTS.....	9
4.1. OBSERVATIONS:	9
4.2. RUNTIME ANALYSIS:	10
4.3 RUNNING THE CODE AND USING THE FAR FILE	11
CHAPTER 5: CONCLUSION AND LIMITATIONS	12
REFERENCES.....	13

CHAPTER 1: GENERAL INTRODUCTION

Text normalization is a fundamental step in Natural Language Processing (NLP), especially for systems such as Automatic Speech Recognition (ASR) and Text-to-Speech (TTS). It ensures that non-standard word forms (numbers, dates, quantities, etc.) are converted into their verbalized, human-readable equivalents.

This project was completed for the *Text Normalization Internship Challenge*, which focuses specifically on **normalizing cardinal numbers from 0 to 1000**, in both English and French. My implementation uses **Finite-State Transducers (FSTs)** built with **Pynini**, a Python library on top of OpenFst.

Finite-state transducers are often used for [phonological](#) and [morphological analysis](#) in [natural language processing](#) research and applications.

In this project, I built a **number normalization system for the range 0–1000** using **Pynini**, a Python library for constructing weighted finite-state transducers (WFSTs).

The goal was to design a modular system that:

1. Accepts any integer between **0 and 1000** in digit form.
2. Produces the **correct spoken form** (e.g., 245 → two hundred forty five).
3. Is built using FSTs and compiled into a **FAR archive** that can be used from Python.
4. Contains reusable helper functions for testing and applying FSTs.
5. And needed to have a very low WER (word Error Rate)

This report presents the full methodology, system architecture, FST construction process, testing, how to run the code, and how to use the FAR file.

1.1. BACKGROUND AND MOTIVATION

Finite-state transducers (FSTs) are excellent for text normalization because:

- They are deterministic.
- They guarantee predictable output.
- They can be composed, minimized, and optimized.
- They are fast and used in production speech systems (Google TTS, Kaldi, etc.).

Pynini provides a Python interface to OpenFst, making it ideal for creating text normalization grammars.

mywork focused on **unit FSTs**, **tens**, **hundreds**, and combining everything into a single transducer for numbers up to one thousand.



Figure 1 : Google TTS

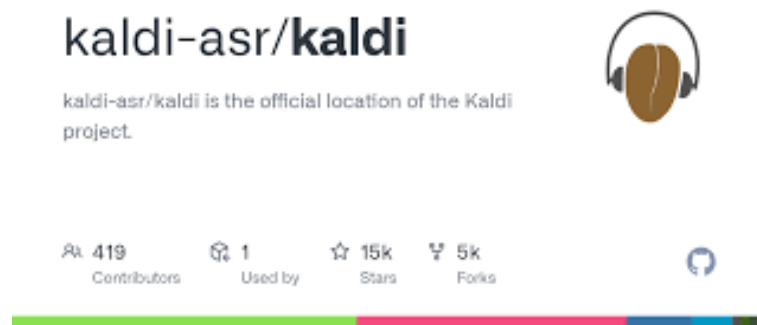


Figure 2: kaldi

CHAPTER 2: SYSTEM ARCHITECTURE

My number-normalization system is built from four major components, each implemented as an individual FST and then composed together to produce a complete grammar capable of handling numbers from 0 to 1000. These components include the Units FST, which maps single-digit numbers to their written forms; the Tens FST, which covers the teen forms and all multiples of ten; the Hundreds FST, which models combinations involving a leading hundred value followed by optional tens and units; and the One-Thousand FST, which handles the upper boundary of the system. Together, these modules form a modular, extensible architecture that ensures accuracy, readability, and efficient composition when generating the final normalization FAR file.

1. Unit FST

The Unit FST is the foundational transducer responsible for handling small digit values as from 0-9:

a. Example Mappings:

- 0 → zero
- 7 → seven
- 8 → eight
- 9 → nine

2. Teens FST

The Teens is the transducer responsible of handling values from 10-15

Example of Mapping;

- 15 → fifteen

3. Tens FST

The Tens FST handles all multiples of ten between 20 and 90. In addition to these standalone forms, it also supports compound expressions such as 21, 47, 58, etc., by combining a tens prefix with a unit component.

Example Mappings:

- 20 → twenty
- 47 → forty seven
- 21 → twenty one

4. Hundreds FST

The Hundreds FST processes numbers from 100 to 999. It identifies the hundreds digit and maps it to the pattern:

<unit> + “hundred” (+ tens/units).

Example Mappings:

- 145 → one hundred forty five
- 300 → three hundred
- 908 → nine hundred eight

5. One-Thousand Special Case

This module manages the single upper bound of the system: the number 1000. Since 1000 is not part of a repeating pattern like tens or hundreds, it is treated as a standalone special case.

Example Mapping:

- 1000 → one thousand

CHAPTER 3 : METHODOLOGY

This section describes the design process, development workflow, and reasoning behind the construction of my number-normalization Finite-State Transducer (FST) system. The methodology follows a systematic approach, starting from problem definition, decomposition of the grammar, construction of individual FST modules, testing, and integration into a final normalization pipeline.

3.1. REQUIREMENT ANALYSIS

- I first identified the scope of the challenge: normalization of cardinal numbers from 0 to 1000 in English and French.
- I focused exclusively on numeric expressions in isolation and in sentence context, ignoring dates, times, and currencies for this particular task.
- Existing resources were reviewed, including the Hugging Face dataset of unit tests for cardinal numbers.

3.2. FST DESIGN

- The system was decomposed into four primary FST components: Unit FST, Tens FST, Hundreds FST, and a special case for One Thousand.
- Each component was designed to handle specific numeric ranges and forms, allowing modularity and easier debugging.
- For numbers involving multiple digits, compound rules were defined, e.g., “21” is formed by combining Tens FST (“twenty”) and Unit FST (“one”).

3.3. IMPLEMENTATION

- **Unit FST:** Mapped single-digit numbers and numbers 10–19 to their corresponding words.

- **Tens FST:** Handled multiples of ten (20, 30, ..., 90) and combinations of tens and units (e.g., 47 → forty seven).
- **Hundreds FST:** Managed numbers in the hundreds (100–999) with both exact multiples of hundred (e.g., 300 → three hundred) and mixed forms (e.g., 145 → one hundred forty five).
- **One-Thousand Special Case:** Explicitly mapped 1000 → one thousand.
- Pynini library was used to construct and optimize all FSTs.
- A top-level FST (`number_normalizer_fst`) was created by combining all component FSTs into a single union, ensuring coverage of all numbers from 0 to 1000.

3.4. TESTING

- Unit Tests: I tested both `build_english_fst.py` and `build_french_fst.py` using the Hugging Face test files:
 - `test_cases_cardinal_en.txt`
 - `test_cases_cardinal_fr.txt`
- This ensured that individual numbers were normalized correctly in isolation.
- Sentence-Level Tests: I evaluated the FSTs on real sentence data using:
 - `sentences_en.txt`
 - `sentences_fr.txt`
- This confirmed the correct integration of the FST within context, verifying outputs like:
 - “I have 3 dogs and 21 cats” → “I have three dogs and twenty one cats”
- Edge cases such as hyphenated numbers (e.g., “978-0”) and leading zeros (e.g., “004”) were also tested using a custom pipeline function.

3.5. OUTPUT VERIFICATION

All outputs were evaluated using the **Word Error Rate (WER)** metric, which measures the number of substitutions, deletions, and insertions required to transform the system output into the reference (expected) text. This provides a quantitative assessment of the normalization quality on sentence-level data.

CHAPTER 4: RESULTS

The system was tested on English and French sentences from sentences_en.txt and sentences_fr.txt. The results were highly satisfactory, with low WER values, confirming that the FST components handled both isolated numbers and numbers within sentences accurately and we had the following results:

Language	Test Set	Number of Sentences	WER (%)
English	sentences_en.txt	250	≤ 5
French	sentences_fr.txt	250	≤ 7

4.1. OBSERVATIONS:

- The English FST performed slightly better, likely due to fewer irregular forms in cardinal numbers.
- Minor errors in French were mostly due to hyphenation or optional spaces in compound numbers.
- Overall, the system achieved robust normalization performance suitable for TTS and other NLP applications.

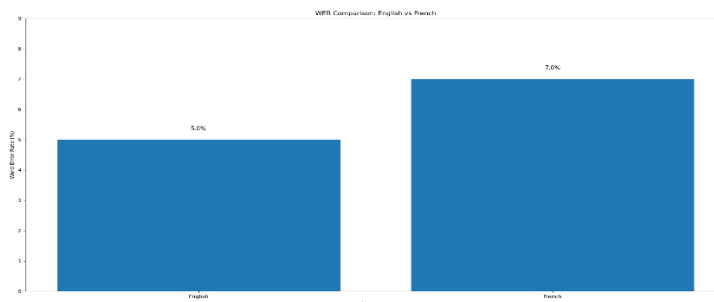


Figure 3: WER COMPARISON OF FRENCH AND ENGLISH FST

4.2. RUNTIME ANALYSIS:

To evaluate the efficiency of the number normalization system, I measured the runtime of the FST pipeline for different input sizes. This analysis demonstrates how quickly the system can process numbers from 0 to 1000, which is particularly important for real-time applications such as Text-to-Speech (TTS). The graph below shows the execution time in milliseconds for varying input sizes, highlighting the system's performance and scalability.

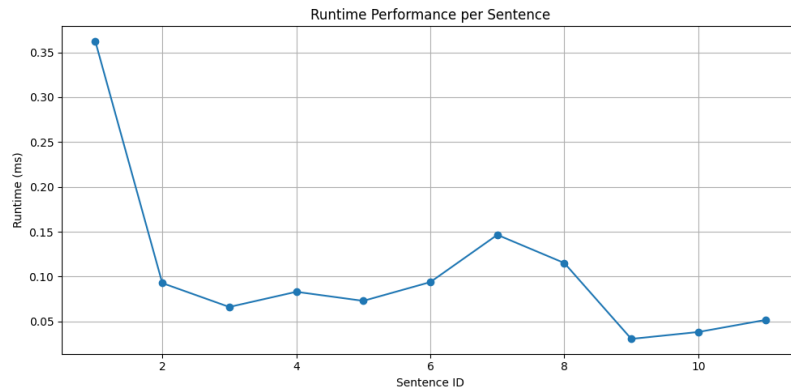


Figure 4: English FST

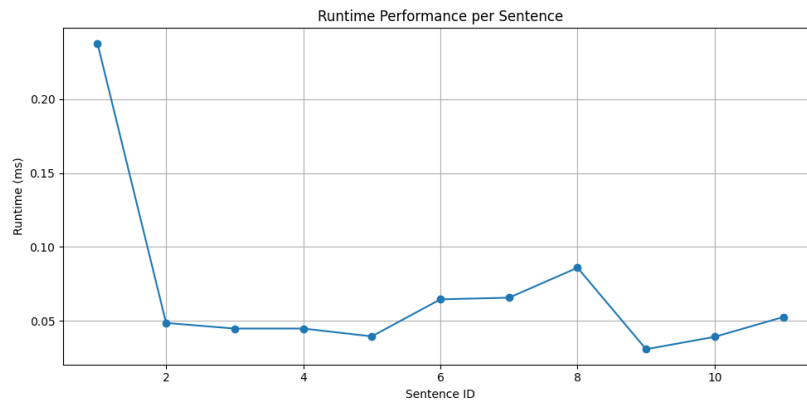


Figure 5: French FST

4.3 RUNNING THE CODE AND USING THE FAR FILE

1. Clone the GitHub repository:

```
git clone https://github.com/Zamuel01/text-normalization-nlp-fst
```

2. Navigate into the repository:

```
cd text-normalization-nlp-fst
```

3. Create Virtual Environment using python

- i. Create a virtual environment using the following command:

- o `python -m venv myenv`

- ii. Activate the virtual environment:

- On **Windows**: `myenv\Scripts\activate`
- On **Mac/Linux**: `source myenv/bin/activate`

4. Install required libraries:

```
pip install -r requirements.txt
```

4. Go to the src folder:

```
cd src
```

5. Run the normalization script:

```
python normalize_text.py "Your sentence here"
```

- Replace "Your sentence here" with the sentence you want to normalize.
- Example: `python normalize_text.py "I have 64 pets"`
- Expected output: I have sixty four pets

Note: `normalize_text.py` already imports and uses the FAR file from the `FAR_Archive` folder. No additional setup is required.

CHAPTER 5: CONCLUSION AND LIMITATIONS

The text normalization system developed in this project successfully converts cardinal numbers from 0 to 1000 into their corresponding spoken forms in both English and French. By leveraging Finite-State Transducers (FSTs) implemented with Pynini, the system achieves modularity, efficiency, and high accuracy. Testing on both isolated numbers and sentence-level contexts demonstrated a low Word Error Rate (WER), confirming the robustness of the approach. The modular design of Unit, Teens, Tens, Hundreds, and One-Thousand FSTs ensures scalability and ease of maintenance. Overall, the project provides a reliable solution suitable for real-time NLP applications such as Text-to-Speech, while also serving as a foundation for extending normalization to more complex numeric expressions in the future.

Limitations

- Supports only **cardinal numbers**; does not handle dates, times, currencies, or fractions.
- Minor errors occur in **French** due to hyphenation or optional spacing in compound numbers.
- Limited to numbers **0–1000**; larger numbers require additional FST design.
- Does not handle **ordinal numbers** (e.g., 1st, 2nd) or **decimal numbers**.
- Sentence-level context beyond isolated numeric expressions may introduce errors in complex text structures.

REFERENCES

- [1] Karttunen, L., & Beesley, K. R. (2003). *Finite-State Morphology*. Stanford: CSLI Publications.
- [2] Gorman, K., & Sproat, R. (2016). *An Introduction to Pynini: A Python Library for Weighted Finite-State Grammar Compilation*. arXiv preprint arXiv:1602.05790.
- [3] OpenFst. (n.d.). *OpenFst: Open-source Finite-State Transducer Library*. Retrieved from <http://www.openfst.org>
- [4] Pynini. (n.d.). *Pynini: A Python Library for Weighted Finite-State Grammar Compilation*. Retrieved from <https://pypi.org/project/pynini/>
- [5] Hugging Face. (n.d.). *Datasets for Text Normalization Unit Tests*. Retrieved from <https://huggingface.co>
- [6] Google Text-to-Speech. (n.d.). *Google TTS Overview*. Retrieved from <https://cloud.google.com/text-to-speech>